Chapter 9

Combination of classifiers

Assoc. Prof. Dr. Duong Tuan Anh
Faculty of Computer Science and Engineering,
HCMC Univ. of Technology
3/2015

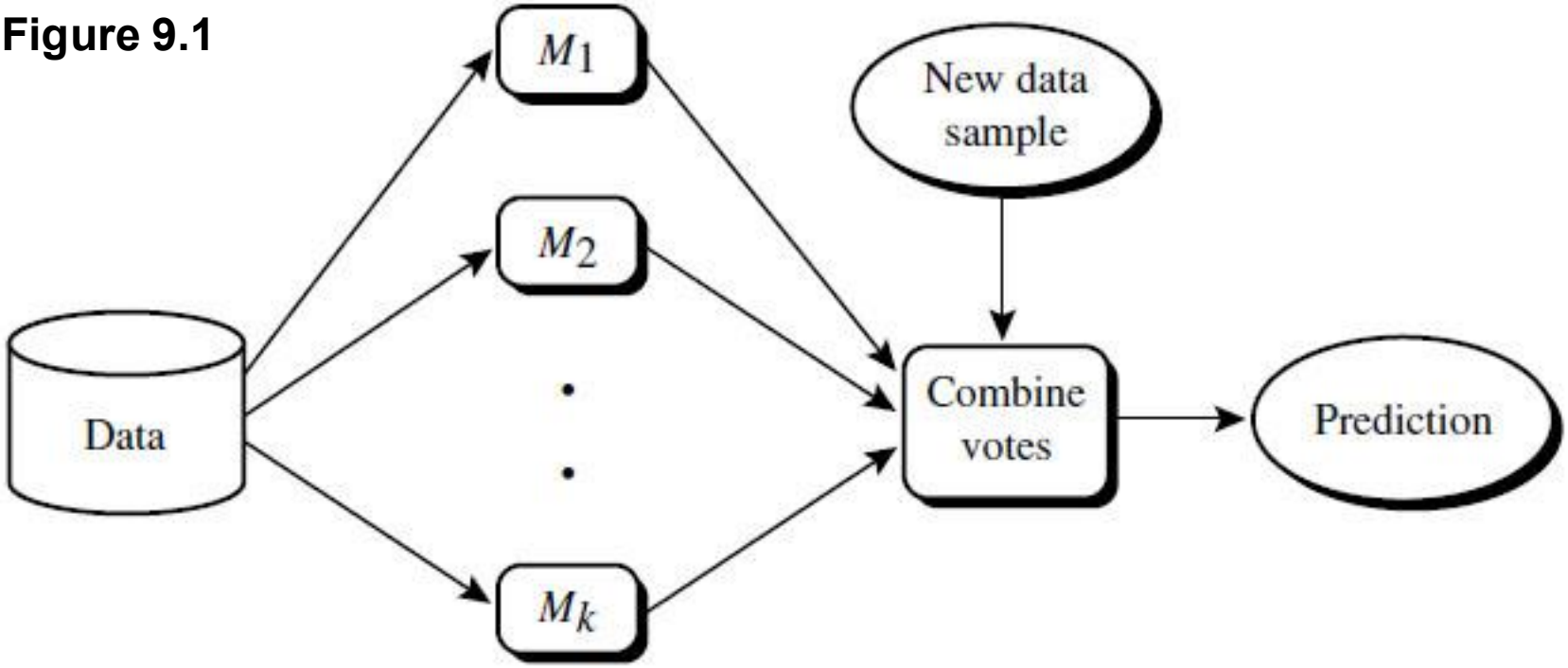
Outline

- 1 Introduction
- 2 Bagging
- 3 Boosting
- 4. Random forest
- 5. ROC curves

Introduction

- A combination or an ***ensemble*** of classifiers is a set of classifiers are combined to classify new examples.
- A combination of classifiers is often more accurate than the individual classifiers that make them up.
- A combination of classifiers is a way of compensating for imperfect classifiers.
- Each classifier is also called an ***expert***.
- ***Bagging*** and ***boosting*** are general techniques that can be applied to classification as well as prediction problems.
- The methods for constructing ensembles of classifiers include
  - Sub-sampling the training set

Figure 9.1



Bagging and boosting each generate a set of classification models, M_1, M_2,…,M_k. Voting strategies are used to combine the classifications for a given unknown sample.

Bagging

- Given a set *D* of *d* tuples, bagging works as follows. For iterations *i* (*i* = 1, 2,…*k*), a training set D_i of *d* tuples is *sampled with replacement* from the original set of tuples, *D*.
- Note that the term bagging stands for ***bootstrap aggregation***.
- Each training set is a bootstrap sample. Because sampling with replacement is used, some of the original tuples of *D* may not be included in D_i, whereas others may occur more than once.
- A classifier model, M_i, is learned for each training set, D_i.

- To classify an unknown tuple, *X*, each classifier, M_i, returns its class prediction, which counts as one ***vote***.
- The bagged classifier, *M**, counts the votes and assigns the class with the most votes to *X*.

Algorithm: Bagging

D: a set of training tuples
k: the number of models in the ensemble.

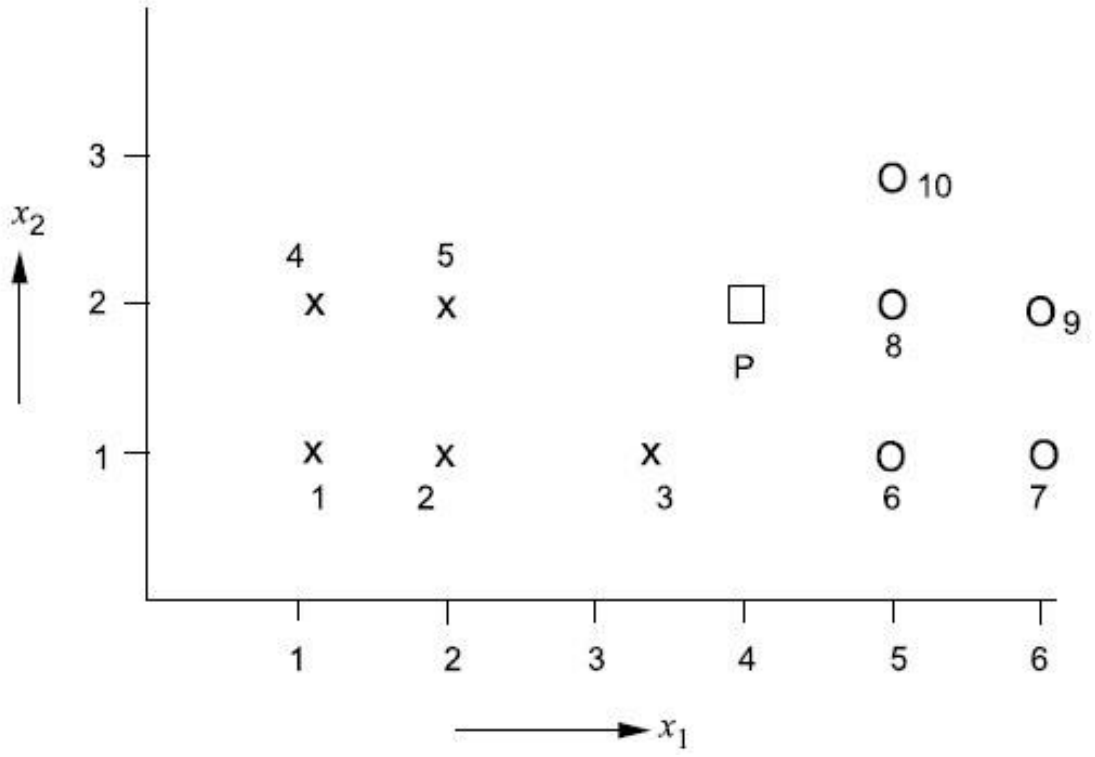
1. **for** *i* = 1 to *k* do // create k models
2. Create bootstrap sample, D_i, by sampling *D* with replacement;
3. Use D_i to derive the model M_i;
4. endfor

To use the composite model to classify tuple, *X*:

1. Let each of the *k* models classify *X* and return the majority vote;

Example

Figure 9.2



Consider the data set given in the figure 9.2 which consists of 10 data points, belonging to 2 classes X and O.

The original training set is divided into a number of disjoint subsets. Then different overlapping training sets are constructed by dropping one of these subsets.

For the dataset in the figure, let us take the disjoint subsets as S_1 = {1, 2}, S_2 = {4, 5}, S_3= {3}, S_4 = {6,7}, S_5 = {8, 10}, S_6 = {9}.

1. If subsets S_1 and S_4 are left out, then the dataset will be {3, 4, 5, 8, 9, 10} and *P* is nearest to O_8 $\Rightarrow$ P is assigned to Class O.
2. If subsets S_1 and S_5 are left out, then the dataset will be {3, 4, 5, 6, 7, 9} and *P* is nearest to X_3 $\Rightarrow$ P is assigned to Class X.
3. If subsets S_1 and S_6 are left out, then the dataset will be {3, 4, 5, 6, 7, 8, 10} and *P* is nearest to O_8 $\Rightarrow$ P is assigned to Class O.
4. If subsets S_2 and S_4 are left out, then the dataset will be {1, 2, 3, 8, 9, 10} and *P* is nearest to O_8 $\Rightarrow$ P is assigned to Class O.
5. If subsets S_2 and S_5 are left out, then the dataset will be {1, 2, 3, 6, 7, 9} and *P* is nearest to X_3 $\Rightarrow$ P is assigned Class X.
6. If subsets S_2 and S_6 are left out, then the dataset will be {1, 2, 3, 6,7, 8,10} and *P* is nearest to O_8 $\Rightarrow$ P is assigned to Class O.

Note: Base classifier = 1-nearest-neighbor

7. If subset S3 and S4 are left out, then the dataset will be {1, 2, 4, 5, 8, 9, 10} and P is nearest to $O_8 \Rightarrow$ P is assigned Class O.
8. If subset S3 and S5 are left out, then the dataset will be {1,2, 4, 5, 6, 7, 9} and P is nearest to $O_6 \Rightarrow$ P is assigned Class O.
9. If subset S3 and S6 are left out, then the dataset will be {1, 2, 4, 5, 6, 7, 8,10} and P is nearest to $O_8 \Rightarrow$ P is assigned to Class O.

The final decision: P is classified as belonging to Class O.

Note: Instead of obtaining independent datasets from the domain, bagging just resamples from the original training set. The datasets generated by resampling are different from each other, but are not independent because they are based on one dataset.

Bagging doesn't work for classifiers that are ***stable***, ones whose output is insensitive to small changes in the input.

Difference between bagging and boosting

- Like bagging, boosting combines models of the same type.
- However, whereas in bagging individual models are built separately, in boosting each new model is *influenced* by the performance of those built previously.
- Boosting encourages new models to become experts for instances classified incorrectly by earlier ones.
- Boosting ***weights*** a model's contribution by its performance, rather than giving equal weight to all models.

Note: The idea of boosting is to "*boost*" weak classifiers into strong classifiers.

Boosting

- In *boosting*, weights are assigned to each training tuple.
- A series of *k* classifiers is iteratively learned.
- After **a classifier M_i** is learned, the weights are updated to allow the subsequent classifier, M_{i+1}, to "pay more attention" to the training tuples that were misclassified by M_i.
- The final boosted classifier M^*, combines the votes of each individual classifier, where the weight of each classifier's vote is a function of its accuracy.

To compute the ***error rate*** of model M_i, we sum the weights of each of the tuples in D_i that M_i misclassified. That is,

$$error(M_i) = \frac{1}{d}\sum_{j}^{d} w_j \, err(X_j) \qquad \textbf{(9.1)}$$

where ***err(X_j)*** is the ***misclassification error of tuple*** X_j: if the tuple was misclassified, the $err(X_j)$ is 1, otherwise, it is 0.

Algorithm: Adaboost

D: a set of training tuples

***k*: the number of models** in the ensemble.

1. Initialize the weight of each tuple in *D* to 1;
2. **for** *i* = 1 to *k* **do** // for each round
3. Sample *D* with replacement according to the tuple weights to obtain D_i;
4 Use training set D_i to derive a model M_i;
5 Compute *error*(M_i), the error rate of M_i
6 **if** *error*(M_i) > 0.5 **then**
7 Reinitialize the weights to 1
8 Go to step 3 and try again
9 **endif**
10 **for** each tuple in D_i that was correctly classified **do**
11 Multiply the weight of the tuple by

$$\beta_i = error(M_i)/(1 - error(M_i)); \quad (9.2)$$

12 Normalize the weight of each tuple
13 **endfor**

To use the composite model to classify tuple, X:

1. Initialize weight of each class to 0;
2. **for** i = 1 to k do // for each classifier
3. $w_i = \log(1/\beta_i)$ // weight for the classifier's vote (log is **ln**)
4. $c_i = M_i(X)$; // get class prediction for X from M_i
5. add w_i to weight for class c_i
6. **endfor**
7. Return the class with the largest weight

Example: Reuse the dataset given in the Figure 9.2 which consists of 10 data points belonging to 2 classes X and O.

Let a weight of 1 be assigned to all the samples, i.e., *weight*(i) = 1, i = 1,..,10.

Consider three classifiers which are based on Hypothesis 1, Hypothesis 2 and Hypothesis 3, respectively.

Classifier 1

This classifier is based on Hypothesis 1. It is that if $x_1 \leq 3$, the pattern belongs to Class X and Class O otherwise. This classifier misclassifies pattern **3**. Which means

error(M_1) = 1/10 = **0.1**

β_1 = 0.1/(1-0.1) = 0.1/0.9 = **0.1111**

weight(1) = 1×0.1111 = 0.1111

Similarly the weights of the other pattern except pattern 3 will be 0.1111. Only the weight of pattern 3 remains as **1**.

Normalizing

weight(1) = 0.1111 × 10/(0.1111 + 0.1111 + 0.1111 + 0.1111 + 0.1111 + 0.1111 + 0.1111 + 0.1111 + 0.1111 + 1) = 0.1111 × 10/(1.9999) =0.5555

weight(2) = 0.5555, weight(4) = 0.5555, weight(5) = 0.5555, weight(6) = 0.5555, weight(7) = 0.5555, weight(8) = 0.5555, weight(9) = 0.5555, weight(10) = 0.5555

weight(3) = 1 × 10/(1.9999) = **5.0002**

Classifier 2

This classifier is based on Hypothesis 2. It is that if $x_1 \leq 5$, the pattern belongs to Class X and Class O otherwise. This classifier misclassifies patterns **6,8** and **10**. Which means

error(M_2) = (1/10) $\times$(0.5555+0.5555+0.5555) = **0.16665**

β_2 = 0.16665/(1-0.16665) = **0.2000**

weight(1) = 0.5555 $\times$0.2 = 0.1111; weight(2) = 0.1111; weight(3) = 5.0002 $\times$0.2 = **1.0004**; weight(4) = 0.1111; weight(5) = 0.1111; weight(6) = 0.5555 $\times$1 = **0.5555**; weight(7) = 0.1111; weight(8) = **0.5555**; weight (9) = 0.1111; weight(10) = **0.5555**;

Normalizing

weight(1) = 0.1111 $\times$ 10/(0.1111 + 0.1111 + 0.1111 + 0.1111 + 0.1111 + 0.1111 + 1.00004 + 0.5555 + 0.5555 +0.5555) = 0.1111 $\times$ 10/(3.33314) =0.333319;

weight(2) = 0.333319,

weight(3) = 1.00004 $\times$ 10/(3.33314) = **3.0003**

weight(4) = 0.333319, weight(5) = 0.333319,

weight(6) = 0.5555 $\times$ 10/(3.33314) = **1.6666**, weight(7) = 0.33319, weight(8) = **1.6666**, weight(9) = 0.333319, weight(10) = **1.6666**;

Classifier 3

This classifier is based on Hypothesis 3. It is that if $x_1 + x_2 \leq 3.5$, the pattern belongs to Class X and Class O otherwise. This classifier misclassifies patterns **3** and **5.** Which means

error(M_3) = (1/10) $\times$(3.0003+0.333319) = **0.3334**

β_3 = 0.3334/(1-0.3334) = **0.5002**

weight(1) = 0.333319 $\times$0.5002 = 0.16673; weight(2) = 0.16673; weight(3) = **3.0003**; weight(4) = 0.16673; weight(5) = **0.333319**; weight(6) = 1.6666 $\times$0.5002 = **0.8336**; weight(7) = 0.16673; weight(8) = **0.8336**; weight (9) = 0.16673; weight(10) = **0.8336**;

Normalizing

weight(1) = 0.16673 $\times$ 10/(0.16673 + 0.16673 + 3.0003 + 0.16673 + 0.333319 + 0.8336 + 0.16673 + 0.8336 + 0.16673 + 0.8336) = 0.16673 $\times$ 10/(6.668069) =0.2502;

weight(2) = 0.2502,

weight(3) = 3.0003 $\times$ 10/(6.668069) = **4.4995**

weight(4) = 0.2502, weight(5) = 0.333319 $\times$ 10/(6.668069)=**0.4999**,

weight(6) = 0.8338 $\times$ 10/(6.668069)= **1.2501**, weight(7) = 0.2502, weight(8) = **1.2501**, weight(9) = 0.2502, weight(10) = **1.2501**;

If we have a test pattern *P* (4,2)

According to classifier 1, it belongs to class O;

According to classifier 2, it belongs to class X;

And according to classifier 3, it belongs to class O.

For Class X,

$\sum \log(1/\beta_i) = \log(1/\beta_2) = \log(1/0.2) =$ **0.699**

For Class O,

$\sum \log(1/\beta_i) = \log(1/\beta_1) + \log(1/\beta_3) = \log(1/0.1111) + \log(1/0.502) =$ **1.2551**

P will be classified as belonging to Class O.

Note: While both ***bagging*** and ***boosting*** can significantly improve accuracy in comparison to a single model, ***boosting*** tends to achieve greater accuracy.

4. Random Forest

- The Random Forest was introduced by Leo Breiman, 2001.
- Random forest integrates a sampling technique (*bootstrap*), a subspace method (*random*) and an ensemble approach (*bagging*).
- Random Forest is appealing because they:
  - Handle both regression and classification.
  - relatively fast to train and classify
  - Can be used for high-dimensional problems.
  - Can easily be implemented in parallel.

Main ideas of Random Forest

- ***Boostrapping*** is applied at the training phase of the Random Forest algorithm.
- Boostrapping is used to generate the training data sets for base classifiers.
- Boostrapping helps generate several subsets from a set of data by randomly selecting the same number of observations as the original dataset, but ***with the replacement***. This allows some patterns from the original data to be *repeated* in a subset of the data set.
- The following figure illustrates sampling by bootstrapping.

Bootstrap

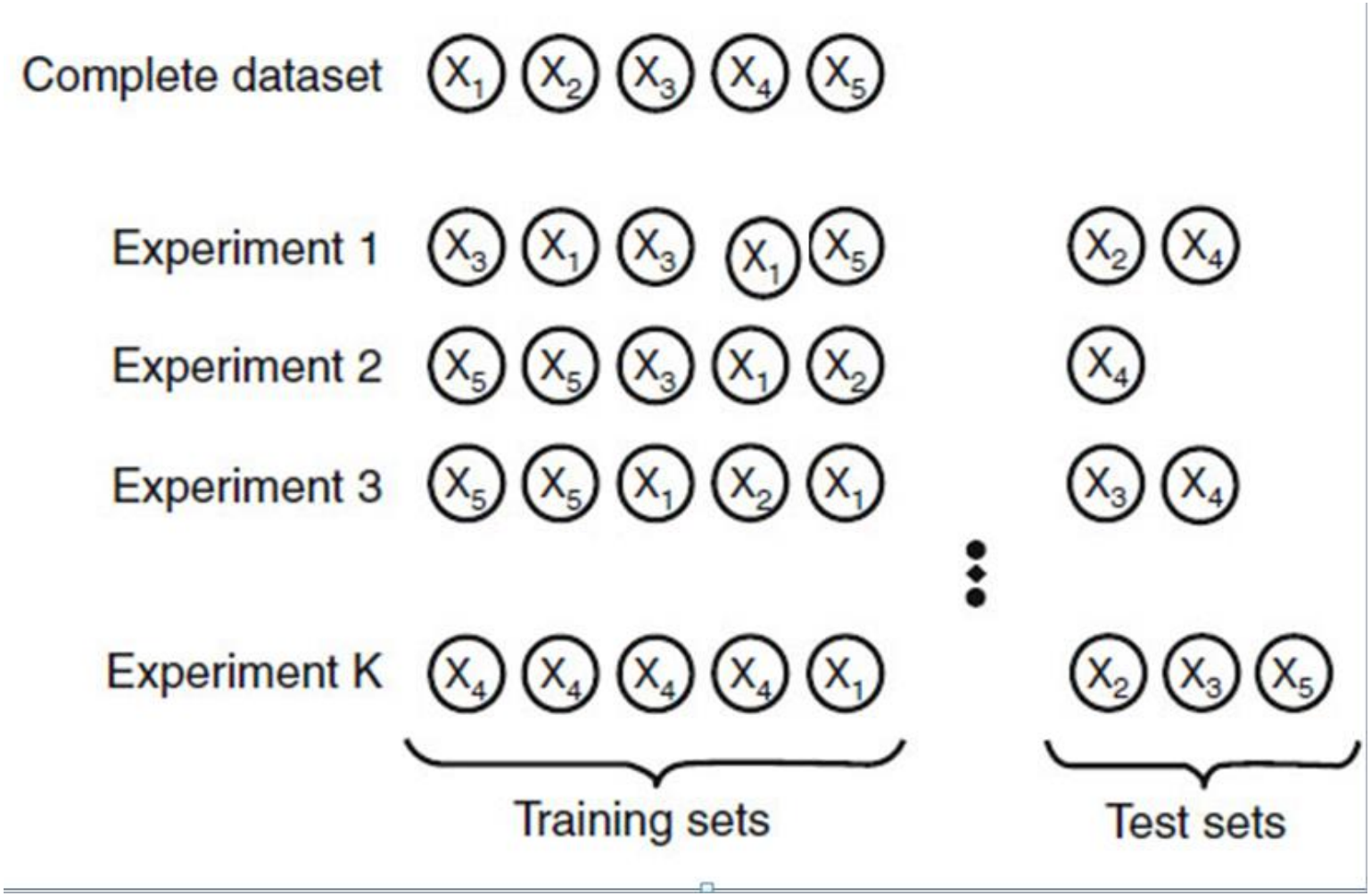


Figure 9.3 The bootstrap method

Main ideas of Random Forest (cont.)

- Base classifiers in Random Forest are ***decision-trees***. Each decision-tree is associated with a randomly selected subspace.
- For example, if the dimension of the space (number of features) of a given data set is p, then we can generate multiple ***subspaces*** with dimension *r*, where $r \leq p$. We may call this a ***subspace division***.
- There are two factors of randomness in Random Forest.
- First, each tree is fit to an independent ***bootstrap sample*** from the original data.
- Second, when splitting a node, the best split is found over a ***randomly selected subset*** of *r* features instead of all *p* features, independently at each node.

Fig. 9.4


All Data
random subset
random subset
random subset
random subset
tree
tree
tree
tree
tree
At each node:
choose some ballsubset of variables at random
find a variable (and a value for that variable) which optimizes the split

Training algorithm for Random Forest

Let $D = \{(x_1,y_1)\ldots,(x_N,y_N)\}$ be the training data with $x_i = (x_{i1},\ldots,x_{ip})$.

For j = 1 to J:

1.Take a bootstrap sample D_j of size N from D.

2. Using the bootstrap sample D_j as the training data, fit the tree using recursive partitioning algorithm.

a. Start with all patterns in a single node.

b. Repeat the following steps recursively for each unsplit node until the stopping criterion is met:

(i) Select r features at random from the p available features.

(ii) Find the best split among all splits on the r features from step (i).

(iii) Split the node into two descendant nodes using the split from step (ii).

Testing algorithm for Random Forest

The testing algorithm consists of two steps:

- *Tree selection*: Suppose we have *J* base classifiers, and then we push the new data according the features through all of these decision trees and obtain its class labels.
- *Bagging*: In this step, the trees selected in the previous step are used to find the *aggregate* of the results obtained.

Note: The bagging suggests the final class label for the new data as the label which is the ***majority*** of the classification results.

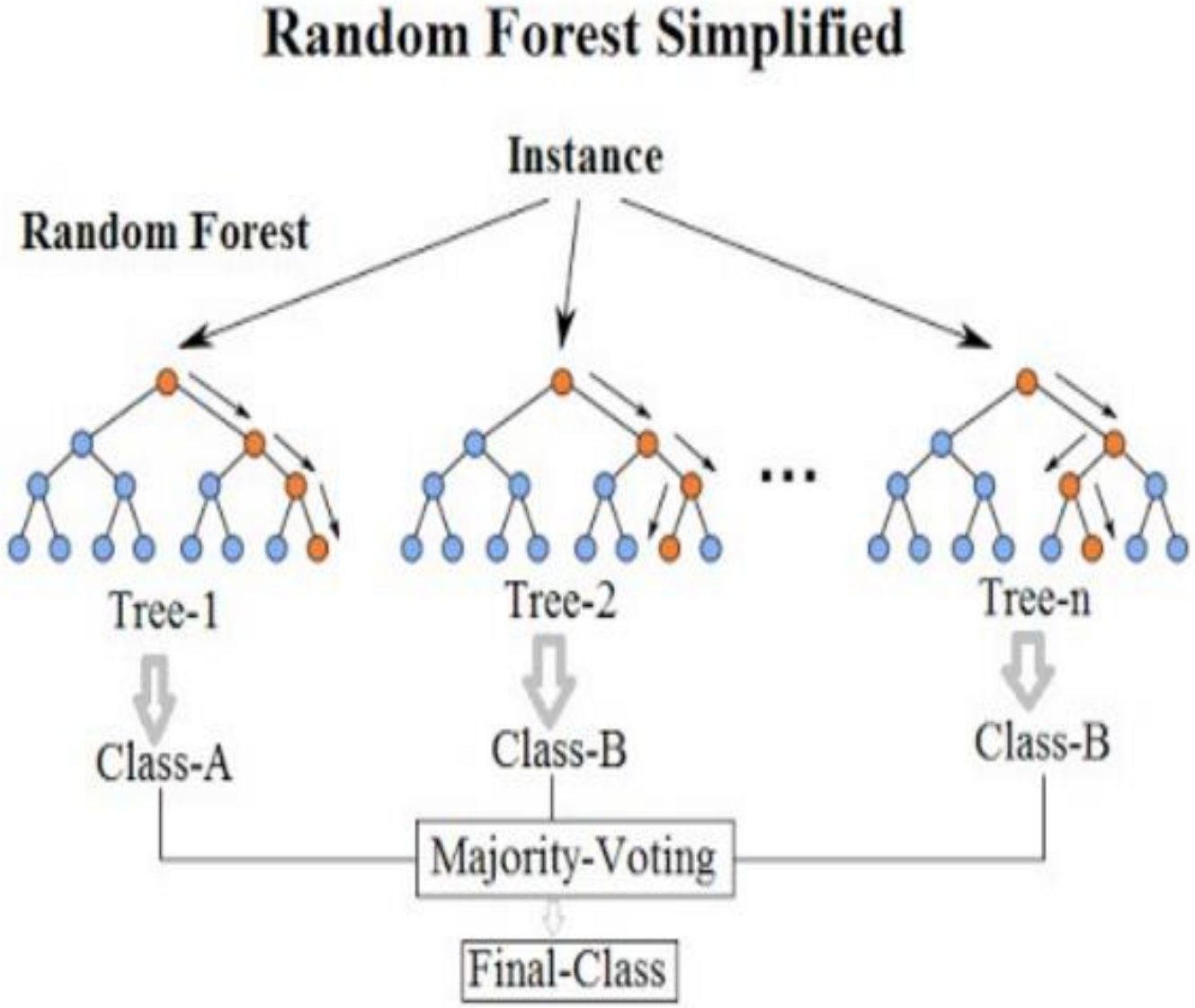
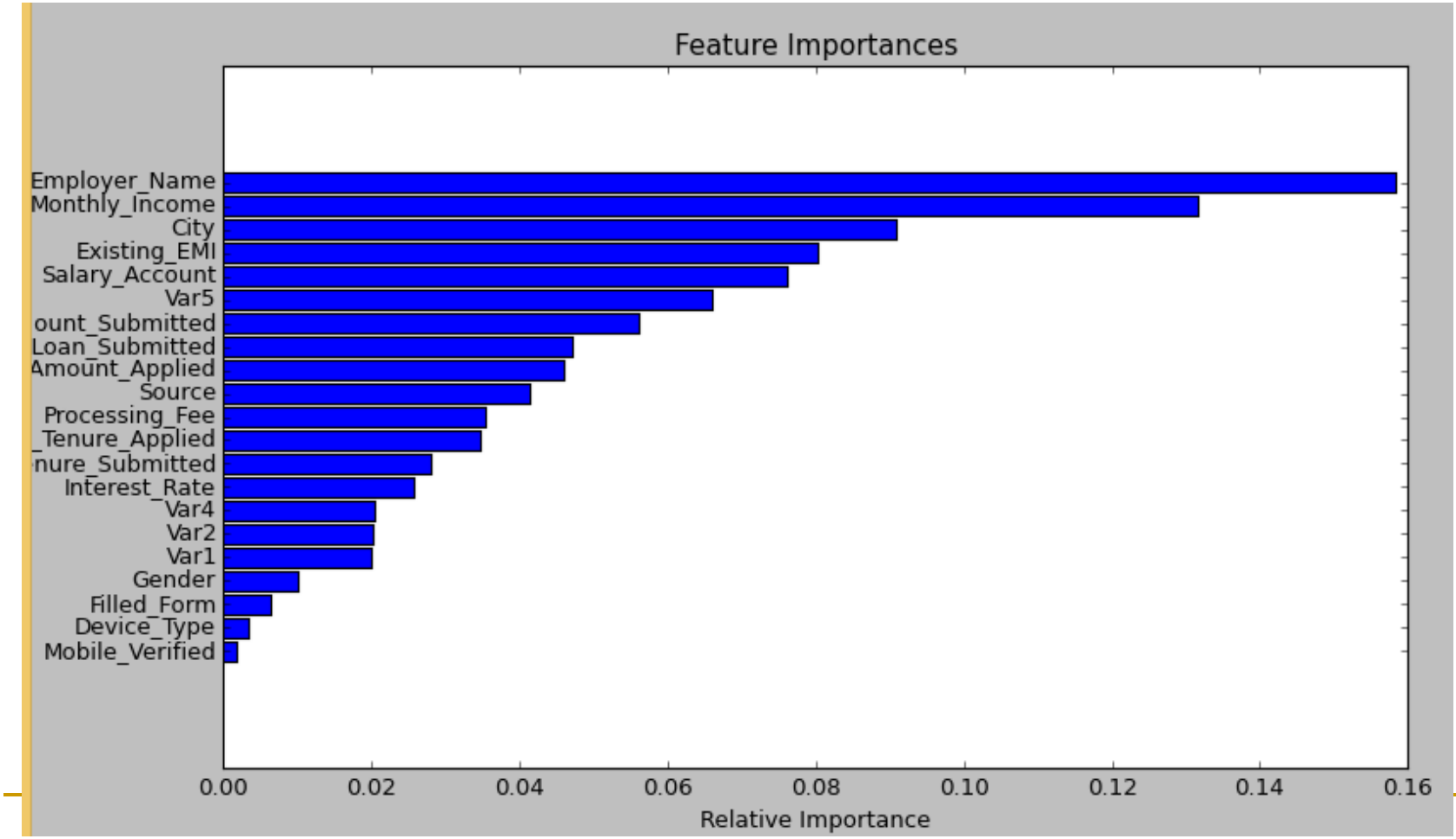

Random Forest Simplified
Instance
Random Forest
Tree-1
Tree-2
Tree-n
Class-A
Class-B
Class-B
Majority-Voting
Final-Class

Figure 9.5

Importance of features

- Random Forest can output the importance of features, i.e. it can identify most important features.



Parameter Tuning

- There are three parameters that may be tuned to given improved accuracy:
  - *r*: the number of randomly selected features chosen at each node.
  - *J*: the number of tree in the forest.
  - *tree size*: as measured by the smallest node size for spitting or the maximum number of terminal nodes.
- The only one of these parameters to which Random Forest is somewhat sensitive appears to be *r*. Some authors suggested that $r \leq \text{sqrt}(p)$.

Empirical Comparison among Ensemble methods

- Base classifier: decision tree
- Each ensemble method consists of 50 decision trees.
- The classification accuracies are obtained from *ten-fold cross-validation.*

- *Datasets:*

  https://archive.ics.uci.edu/ml/datasets.html

Table 5.5. Comparing the accuracy of a decision tree classifier against three ensemble methods.

Data Set	Number of (Attributes, Classes, Records)	Decision Tree (%)	Bagging (%)	Boosting (%)	RF (%)
Anneal	(39, 6, 898)	92.09	94.43	95.43	95.43
Australia	(15, 2, 690)	85.51	87.10	85.22	85.80
Auto	(26, 7, 205)	81.95	85.37	85.37	84.39
Breast	(11, 2, 699)	95.14	96.42	97.28	96.14
Cleve	(14, 2, 303)	76.24	81.52	82.18	82.18
Credit	(16, 2, 690)	85.8	86.23	86.09	85.8
Diabetes	(9, 2, 768)	72.40	76.30	73.18	75.13
German	(21, 2, 1000)	70.90	73.40	73.00	74.5
Glass	(10, 7, 214)	67.29	76.17	77.57	78.04
Heart	(14, 2, 270)	80.00	81.48	80.74	83.33
Hepatitis	(20, 2, 155)	81.94	81.29	83.87	83.23
Horse	(23, 2, 368)	85.33	85.87	81.25	85.33
Ionosphere	(35, 2, 351)	89.17	92.02	93.73	93.45
Iris	(5, 3, 150)	94.67	94.67	94.00	93.33
Labor	(17, 2, 57)	78.95	84.21	89.47	84.21
Led7	(8, 10, 3200)	73.34	73.66	73.34	73.06
Lymphography	(19, 4, 148)	77.03	79.05	85.14	82.43
Pima	(9, 2, 768)	74.35	76.69	73.44	77.60
Sonar	(61, 2, 208)	78.85	78.85	84.62	85.58
Tic-tac-toe	(10, 2, 958)	83.72	93.84	98.54	95.82
Vehicle	(19, 4, 846)	71.04	74.11	78.25	74.94
Waveform	(22, 3, 5000)	76.44	83.30	83.90	84.04
Wine	(14, 3, 178)	94.38	96.07	97.75	97.75

5. ROC curves

- ROC curves are a useful visual tool for comparing two classification models.
- ROC stands for *Receiver Operating Characteristic*.
- ROC curve is a plot of the *true positive fraction* (TPF) against the *false positive fraction*, FPF.
- TPF: proportion of positive tuples that are correctly identified. FPF: proportion of negative tuples that are incorrectly identified as possitive.
- The vertical axis of an ROC curve represents the ***true positive fraction***. The horizontal axis of an ROC curve represents the ***false positive fraction***.
- To plot an ROC curve for a given classification model, M, the model must be able to return a probability for the predicted class of each test tuple. That is, we need to rank the test tuples in ***decreasing order***, where the one the classifier thinks is most likely to belong to the positive class appears at the top of the list.

How to plot ROC curve

An ROC curve is plotted as follows.

- Starting at the bottom-left hand corner (where the TPF and FPF are both 0), we check the actual class label of the tuple at the top of the list. If we have a true positive then on the ROC curve, we move up and plot a point. If, the tuple really belongs to the ‘no’ class, we have a false positive, on the ROC curve, we move right and plot a point.
- The process is repeated for each of the test tuples, each time moving up on the curve for a true positive or toward the right for a false positive.

The figure 9.6 shows the ROC curves of two classification models. *The closer the ROC curve of a model is to the diagonal, the less accurate the model.*

To assess the accuracy of a model, we can measure *the area under the curve*. The closer the area is to 0.5, the less accurate the model is. A model with perfect accuracy will have an area of 1.

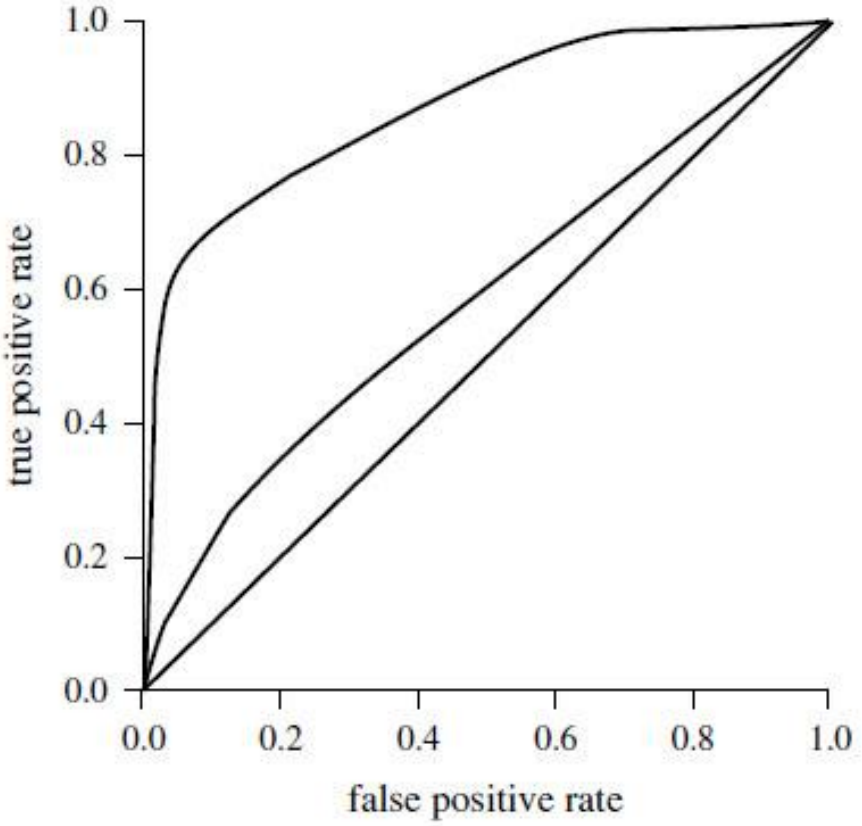


Figure 9.6 The ROC curves of two classification models.

Several software packages support the plot of ROC curves and compute the area under the curves (e.g. MATLAB)

Tools for bagging, boosting and Random Forest

- Bagging and ADABOOST are available in WEKA software.
- ADABOOST is available in R.
- Random Forest is available in WEKA, R.
- Random Forest is available in scikit-learn

Note: ADABOOST is one of top-ten algorithms in Data mining.

References

- M. N. Murty and V. S. Devi, 2011, *Pattern Recognition – An Algorithmic Approach*, Springer.
- J. Han and M. Kamber, *Data Mining: Concepts and Techniques*, 2nd Edition, Morgan Kaufmann Publishers, 2006.
- A. Cutler, D.R. Cutler and J. R. Stevens, *Random Forests*, Chapter 5, in: Ensemble Machine Learning: Methods and Applications, C. Zhang and Y. Ma (eds.), Springer, 2012.

Terminology

- Ensemble of classifiers: *tổ hợp bộ phân lớp*, bagging, boosting, bootstrap sampling: *lấy mẫu kiểu bootstrap*, aggregation: *gộp*, individual classifier: *bộ phân lớp thành phần*, base classifier: *bộ phân lớp cơ sở,* random forest: *rừng ngẫu nhiên*, subspace: *không gian con*, vote strategy: *chiến lược bầu phiếu*, majority vote: *tính phiếu theo đa số*, ROC curve: *đường cong ROC*.